

ARCHITECTURE DESIGN SPECIFICATION ·
TECHNICAL RFC

LionFS

2.0

The target architecture for a line-rate, media-aware, self-healing universal file system — grounded in the measured LionFS 1.x baseline, where every performance claim is reproducible.

LionFS Project

Architecture Review Board · September 2026

LFS-RFC-002 · STATUS: PROPOSED

Table of Contents

1. Baseline: LionFS 1.x Today	1
1.1 What exists and what it is made of	1
1.2 Measured behavior (all numbers reproducible)	2
1.3 Structural gaps the 2.0 architecture must close	3
2. Design Goals, Workloads and Success Metrics	4
2.1 Target hardware and the line-rate budget	4
2.2 Workload matrix and success metrics	4
2.3 Non-goals, stated plainly	5
2.4 Latency and durability service levels	5
3. Pillar I: I/O Engine and Concurrency Model	6
3.1 The submission path: io_uring as the only front door	6
3.2 The bypass plane: SPDK and DPDK where every nanosecond is billed	8
3.3 The lock-free core	8
3.4 Tiered and direct memory access: CXL and DMA	9
4. Pillar II: Addressing, Namespace and File Specialization	9
4.1 The 128-bit volume address space	9
4.2 Small files: inline in the metadata, tail-packed to the last byte	10
4.3 Large files: the B-epsilon extent index	11
5. Pillar III: Reliability, Self-Healing and Crash Recovery	12
5.1 Crash consistency: redirect-on-write plus intent journal	12
5.2 Dual-speed checksums and continuous scrubbing	12
5.3 Autonomous repair	13
5.4 The recovery state machine	13
6. Pillar IV: Hardware-Aware Tiering and Alignment	14
6.1 Media policies	14
6.2 Universal alignment guarantees	15

7. Pillar V: Compression and Deduplication Pipeline	15
7.1 Tiered adaptive compression	15
7.2 Inline deduplication with content-defined chunking	16
7.3 Ratio accounting and the honesty rule	17
8. Deliverable A: Core Architecture Blueprint	17
8.1 On-disk layout	18
8.2 Allocation structures	20
8.3 Journaling and copy-on-write trees	20
9. Deliverable B: Read/Write Lifecycle Walkthrough	21
9.1 Reading 1 MiB (warm file, compressed pool)	21
9.2 Writing 1 MiB (transactional, RAID5 pool)	23
9.3 Variants that matter	25
9.4 fsync, group commit, and the flush path	25
9.5 Failure injection and proof obligations	26
10. Deliverable C: Trade-off Analysis	26
11. Implementation Roadmap	28
11.1 Normative references	30

Executive Summary

LionFS 2.0 is the target architecture for a line-rate, media-aware, self-healing universal file system. It is not a clean-room fantasy: it is a forward specification grounded in a real, running codebase. LionFS 1.x already implements an extent-based FUSE file system in Rust with a versioned on-disk format, RAID 0/1/5/6/10 pools with incremental parity, compression clusters, a per-block checksum tree, and a transaction journal - and every performance claim about it in this document comes from measured, reproducible runs of the in-process **lfs_ioperf** harness, not from marketing. This RFC keeps that honesty rule as a first-class design constraint: no number appears here that was not produced by a command a reader can re-run.

The 2.0 architecture is organized around five pillars, each answering a failure mode we can observe in the 1.x profile. Pillar I rebuilds the I/O engine on **io_uring**, with an optional SPDK/DPDK kernel-bypass plane, per-core lock-free sharding, and RCU path lookup, so that no lock, syscall, or buffer copy sits between an application and a PCI Express 5.0 device saturating at roughly 14 GB/s. Pillar II moves the namespace to **128-bit addressing** with packed 16-byte extent records, inlines files under 4 KiB directly into metadata leaves with dynamic tail packing, and replaces the extent-spill B-tree with a write-optimized B-epsilon tree. Pillar III hardens the proven redirect-on-write transaction pipeline into zero-fsck crash recovery and adds continuous online scrubbing with autonomous erasure repair. Pillar IV makes placement media-aware: zone-append for ZNS devices, band-sequential writes for SMR, and probed 4K/16K/64K alignment everywhere. Pillar V upgrades the compression cluster scheme to per-cluster BLAKE3 integrity, tiered LZ4/Zstd pipelines with hardware offload, and adds inline deduplication for cold pools.

5	3	7
architecture pillars	required deliverables	roadmap phases (P0-P6)

Three deliverables are required by the review board and provided in full: a **core architecture blueprint** with field-level on-disk structure tables (Section 8), a **read/write lifecycle walkthrough** that traces operations from user space down to NAND and magnetic media (Section 9), and a **trade-off analysis** that confronts the classical operating-system tensions - copy-on-write fragmentation versus in-place write speed, metadata memory footprint versus lookup latency, compression ratio versus CPU - and states the mechanism, the cost, and the residual risk of every resolution (Section 10). Section 11 maps the whole target onto the existing codebase as a seven-phase roadmap with exit criteria, each phase benchmarked under the same interleaved A/B protocol the 1.x work used.

The ask of the architecture review board is narrow: approve the 2.0 target architecture and the P0-P6 sequencing. The design deliberately reuses proven 1.x substrate - the 32-byte cluster record, the GF(256) parity engine, the transaction manager, the upgrade path - so that each phase lands as a measured,

reversible delta rather than a rewrite.

1. Baseline: LionFS 1.x Today

1.1 What exists and what it is made of

LionFS 1.x is a Rust userspace file system mounted through **fuser 0.12** (pure-Rust FUSE bindings, ABI 7.10). The on-disk format is at version 2: 4 KiB blocks, the LIONFS10 magic, a checksummed superblock with three copies, and root pointers for the inode tree, directory tree, extent tree, free-space tree, checksum tree, snapshot, clone, refcount, subvolume, dedup, key, device, and block-transform trees. Inodes are 256 bytes with seven inline extent slots; the eighth extent spills into a per-inode B-tree. Version 2 added the feature this RFC builds upon most heavily: **compression clusters**, where 32 logical blocks (128 KiB) are compressed as a unit into a variable-length physical extent, mapped by a per-inode ClusterTree keyed by cluster index. That single design decision - proven in production code - is what makes compression in LionFS actually save space rather than merely reduce entropy.

The storage layer pools devices into RAID 0/1/5/6/10 with a GF(256) parity engine and an incremental read-modify-write path for P and Q syndromes, with full-row recompute retained as the replay-safe fallback. Integrity is enforced by a per-block XxHash64 checksum tree, plus BLAKE3, SHA-256, and CRC32 primitives in the integrity module. Encryption (AES-256-GCM, ChaCha20-Poly1305) rides a per-block transform tree of nonces and tags. Background workers handle flushing, scrubbing, garbage collection, rebalancing, and optimization. Table 1 inventories the subsystems this RFC will touch, mapped to the 1.x modules that already implement their skeletons.

Subsystem	1.x module(s)	State	2.0 role
I/O entry	src/fuse, src/mount	Working, fuser round-trip	Replaced by io_uring daemon; FUSE kept as compat
Block I/O	src/disk (block_io, geometry, sectors)	Sync read/write, zero-copy paths	Async engine, batched rings, geometry probed
Allocator	src/allocator (bitmap, locality, extents, policies)	Bitmap + speculative sizing, per-file locality	Per-core sharded free-space queues
Extent index	src/btree, src/extents	Plain B-tree spill, merge/split	B-epsilon tree with padded inserts
Transactions	src/transaction (manager, commit, checkpoint, rollback)	Journal + checkpoint pipeline	Same shape, intent-log hardening
RAID / erasure	src/pool (raid, gf256, recovery)	RAID 0/1/5/6/10, incremental RMW	Kept; SIMD GF(256), heal path to scrubber
Integrity	src/integrity (checksum_tree, bad_blocks)	XxHash64 tree, scrub tooling	Dual-speed checksums + autonomous repair

Subsystem	1.x module(s)	State	2.0 role
Compression	src/fs/compression, src/file/cluster	128 KiB clusters, ClusterTree, zstd level mount option	Kept; per-cluster BLAKE3, tiered, QAT
Caches	src/cache (inode, extent, dir, node), moka	moka LRU caches	RCU-protected, per-shard instances
Tools	tools/ (ioperf, mkfs, upgrade, scrub, ...)	30+ CLI binaries, lfs_ioperf harness	Benchmark protocol carried forward

Table 1: 1.x subsystem inventory relevant to the 2.0 target

1.2 Measured behavior (all numbers reproducible)

The development container has no /dev/fuse, so 1.x numbers come from lfs_ioperf, an in-process harness that drives the real I/O core - FileManager, allocator, checksum tree, transaction layer, RAID engine - against tmpfs images. They measure userspace CPU cost, not device throughput, and are explicitly not comparable to fio-on-mount numbers. Every before/after comparison was produced as interleaved A/B runs, three rounds, medians, because single runs drift up to 15 percent on the shared 2-vCPU host. Within those rules, the 1.x upgrade program delivered the results in Table 2, and the extent-fragment column is the honest star of the table: speculative extent sizing plus metadata zoning took a sequentially written 32 MiB file from 8,192 fragments to 8.

Pattern	Baseline (P0)	Final (P5)	Delta	Fragments
seq4k-write-fresh	528 MiB/s	569 MiB/s	+7.7%	8192 to 8
seq4k-write	882 MiB/s	1033 MiB/s	+17.1%	7
seq4k-read	1194 MiB/s	1441 MiB/s	+20.7%	7
seq64k-write-fresh	561 MiB/s	832 MiB/s	+48.4%	8192 to 8
seq64k-write	930 MiB/s	1060 MiB/s	+14.0%	7
seq64k-read	1197 MiB/s	1454 MiB/s	+21.4%	7
rand4k-read	1120 MiB/s	1323 MiB/s	+18.1%	8
rand4k-write	824 MiB/s	831 MiB/s	+0.8%	8191 (expected)

Table 2: 1.x single-device results, baseline vs final (lfs_ioperf medians)

+71.0% RAID5 6-dev rand 4K commit, incremental parity (629 vs 367 MiB/s)	2.90x compression ratio at 407 MiB/s write, zstd level 3	8192 to 8 extent fragments for a 32 MiB sequential file
--	---	--

The negative results are as load-bearing as the positive ones. Markov readahead measured minus 48 to minus 51 percent on every read pattern in the harness - the per-read LRU insert reintroduced the 4 KiB copy that zero-copy had removed - so it ships default-off, guarded by an environment flag. Random 4 KiB writes are honestly flat at +0.8 percent because first-touch random writes allocate one extent per block by design, and the checksum-tree insert, which a no-checksum comparison showed dominates roughly 45 percent of write cost, was not on the optimization path. Compressed random 4 KiB writes pay a whole-cluster read-modify-write, the same class of trade-off Btrfs makes. Section 1.3 turns each of these into an explicit 2.0 design requirement instead of leaving them as footnotes.

1.3 Structural gaps the 2.0 architecture must close

Read together, the measurements say the remaining distance to line rate is not in any single algorithm; it is structural. Eight properties of the 1.x architecture cap the achievable throughput, and each maps to a pillar of this specification. The FUSE round-trip and per-call buffer copies put two context switches and two copies on every operation; synchronous std I/O serializes the engine against device latency; the checksum insert sits on the write path; small files burn a full 4 KiB block plus a metadata block for a few hundred bytes of payload; and the media knows nothing about zones, bands, or erase blocks. Table 3 is the contract this RFC answers to - the right column is filled by the sections that follow.

Gap	Root cause in 1.x	2.0 answer	Section
Syscall + copy per I/O	fuser round trip, per-call Vec buffers	io_uring rings, registered zero-copy buffers	3
Serialized device waits	sync std::fs read/write in block_io	Fully async engine, batched completion	3
Global hot-path locks	single engine, shared mutable state	Per-core shards, RCU walks, seqlocks	3
Checksum insert cost (approx. 45% of write)	synchronous checksum-tree insert per block	Deferred verify on completion; B-epsilon inserts	3, 4, 5
4 KiB minimum per file	one-block allocation granularity	Inline data in inode leaves, tail packing	4
Spill B-tree fanout	plain B-tree, 7 inline slots then tree	B-epsilon extent index, 16 B records	4
Whole-cluster RMW on compressed rand write	128 KiB cluster is the write unit	Punch-through escape hatch, tiered codecs	7
No ZNS / SMR / CXL awareness	device treated as flat block array	Zone-append, band-sequential, PMEM tier	6

Table 3: Gap analysis - observed 1.x limit to 2.0 mechanism

2. Design Goals, Workloads and Success Metrics

2.1 Target hardware and the line-rate budget

The design target is a dual-socket x86-64 or ARM server with one or more PCIe 5.0 x4 NVMe devices, scaling to PCIe 6.0 and CXL-attached memory in the later roadmap phases. A PCIe 5.0 x4 link moves roughly 15.7 GB/s in one direction after encoding; current flagship controllers sustain 12 to 14 GB/s sequential and one to two million 4 KiB IOPS at high queue depth. Saturating that from a file system is an arithmetic problem before it is a software problem: at 2 million IOPS across 16 cores, the per-core budget is 8 microseconds per operation, and the file system is allowed to consume a minority of it after the kernel's own I/O stack takes its share. That is why the architecture is fanatical about per-operation fixed costs - a syscall is a few hundred nanoseconds, an unnecessary 4 KiB copy at 14 GB/s is 300 nanoseconds of pure bandwidth, and a global lock is unbounded. Table 4 states the budget the engine must fit.

Quantity	Value	Note
Device sequential rate	14 GB/s	flagship Gen5 x4 controllers
Device random 4 KiB rate	2.0M IOPS at QD 256	aggregate, both dies
Per-core op budget (seq)	1.1 microsec per 4 KiB op	16 cores, 60% engine share
Per-core op budget (rand)	4.8 microsec per op	16 cores, 40% engine share
Syscall cost to avoid	0.1-0.3 microsec each	io_uring amortizes to zero
4 KiB copy to avoid	0.3 microsec each	registered buffers, DMA to user
Global lock budget	0	any shared cacheline is a ceiling

Table 4: Per-operation CPU budget at line rate (16-core host, Gen5 x4 device)

2.2 Workload matrix and success metrics

Six workload classes cover the intended deployment envelope, from database and analytics I/O to immutable backup pools. Each class has a target the roadmap must hit before the phase that delivers it is considered done, measured with the same interleaved A/B median protocol as the 1.x program and, from P0 onward, with fio against a mounted file system on real NVMe - the harness that was impossible in the 1.x container becomes mandatory once the io_uring daemon lands.

Workload class	Representative profile	Target
Large sequential streaming	64 KiB-1 MiB read/write, QD 32+	at least 90% of device line rate
Random 4 KiB point I/O	fio randread/randwrite, QD 64-256	at least 2x 1.x rand4k write; 85% of raw device
Metadata-heavy traversal	open/close/stat of 10M files	10x 1.x via RCU cache + HAMT
Small-file CRUD	sub-4 KiB files, tar/untar corpus	1 metadata read per file; 0 data blocks
Compressed cold storage	mixed corpus, zstd cold tier	ratio at least 2.5x at 3 GB/s write
fsync-heavy transactions	32 KiB append + fsync, 8 writers	group commit; 1 device flush per batch

Table 5: Workload classes and 2.0 success targets

2.3 Non-goals, stated plainly

A specification that claims everything proves nothing. LionFS 2.0 explicitly does not target the following, each for a reason the review board should hold it to. First, there is no distributed consensus or multi-node locking: the replication plane is an asynchronous, snapshot-shipping hook, because coherent clustering changes failure modes this RFC does not budget for. Second, the SPDK bypass plane deliberately gives up kernel page-cache sharing and full POSIX semantics for other processes on the same devices - bypass mode is opt-in per pool, not a global default. Third, no cross-file-system comparison number will appear in LionFS documentation unless ext4, XFS, Btrfs, or ZFS was actually built, mounted, and measured on the same hardware in the same run; the 1.x repo removed a fabricated comparison table once already, and this RFC carries that scar forward as a rule. Fourth, 256-bit dynamic addressing is analyzed and rejected in Section 10 rather than silently ignored: no shipping or announced medium reaches the bounds of the 128-bit volume namespace, and wider keys measurably hurt lookup and cache line density.

2.4 Latency and durability service levels

Throughput targets alone let a file system win benchmarks and lose applications, so the specification fixes three latency and durability service levels that every phase must hold while it chases line rate. The read SLO is the tightest: a warm-cache 4 KiB read must complete in no more than 10 microseconds of engine time, because database working sets live or die by point-lookup tails, and every mechanism in Pillars I and II - RCU walks, shift-split resolution, deferred checksums - exists to keep that tail out of the scheduler's hands. The fsync SLO bounds group commit: at 64 concurrent fsyncing writers, the engine must coalesce into one device flush per journal transaction batch, which the intent-log design of Pillar III makes structural rather than opportunistic. The recovery SLO caps mount-after-crash at the time to replay the intent log - seconds, not minutes, and never a full-volume scan. Each SLO has a named counter in the health bus and a slot in the P0 benchmark protocol, so a phase that improves bandwidth while breaking an SLO fails its exit criteria explicitly rather than shipping a regression quietly.

- Warm 4 KiB read: at most 10 microseconds engine time, p99, measured on the ring.
- fsync at 64 writers: one device flush per group-commit batch; at most 2 per second residual.
- Crash recovery: mount completes in $O(\text{journal})$; target under 5 seconds at 256 MiB log.
- Scrub and GC: bounded to 5 percent of device bandwidth, foreground-latency neutral.

3. Pillar I: I/O Engine and Concurrency Model

3.1 The submission path: `io_uring` as the only front door

Every synchronous boundary is a throughput ceiling. The 1.x engine enters through FUSE, pays a kernel round trip per operation, and copies the payload twice; its block layer then calls synchronous read/write, serializing the whole engine against device latency. The 2.0 front door is a per-application **`io_uring`**: the daemon mmaps the submission and completion queues into the client, batches of SQEs are produced lock-free, and registered buffers let the device DMA land results directly in application memory. In steady state the submission path performs no syscalls at all - SQPOLL mode keeps a kernel thread consuming the ring, and the doorbell is required only when the ring transitions from empty to non-empty. Interrupt-driven completions are replaced by IOPOLL where the device supports it, trading a bounded amount of busy polling for interrupt-eliminated latency on the hot core. Figure 1 shows the four phases of the pipeline; the full step-by-step trace with per-step costs is Section 9.

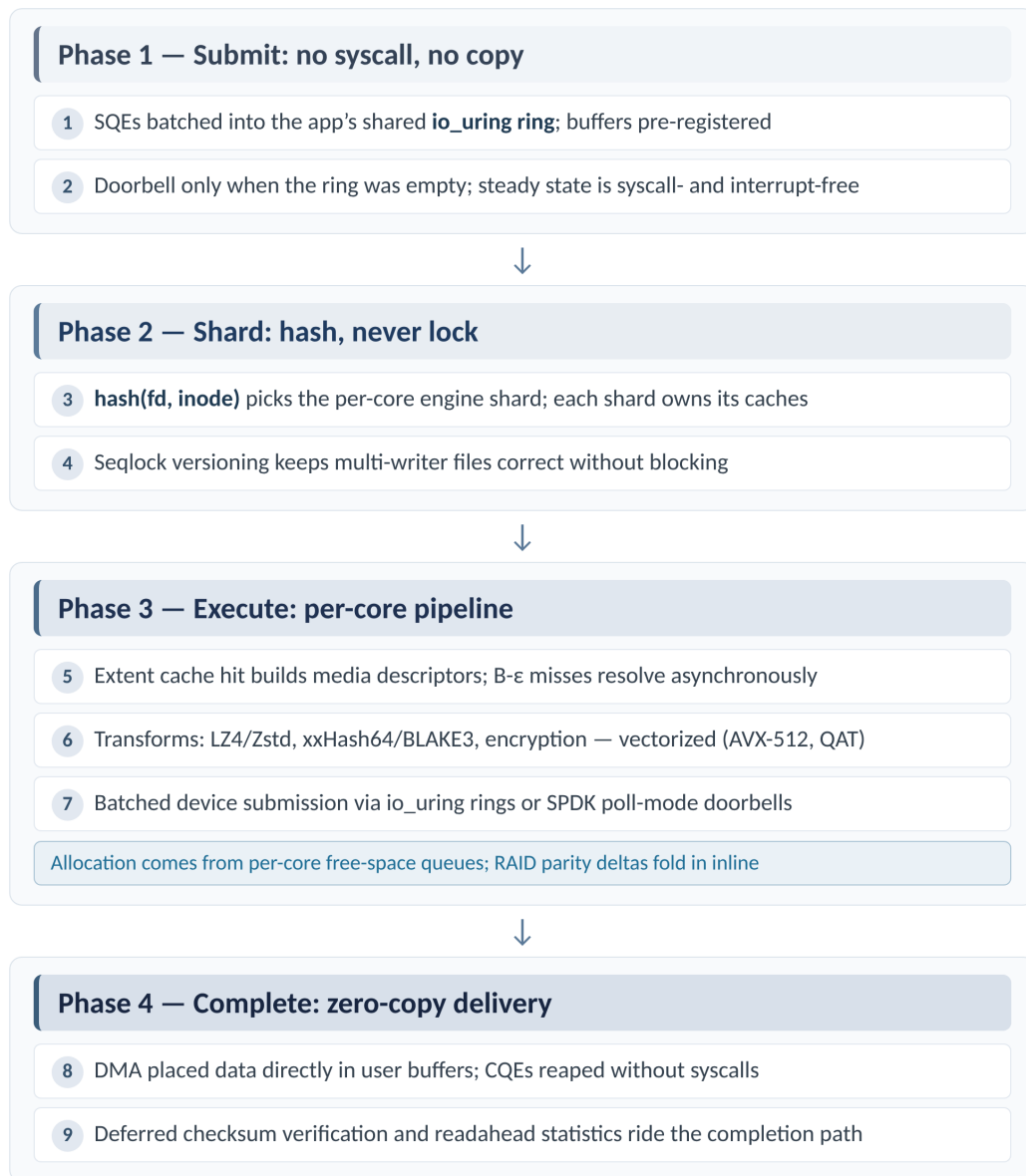


Figure 1: I/O engine pipeline - submit, shard, execute, complete

Parameter	Default	Rationale
Submission queue depth	1024 entries	covers QD 64 x 16 cores without overflow
Completion queue depth	4 x SQ	absorbs completion bursts without drops
Registered buffer regions	128 x 2 MiB	large enough for streaming, small enough to pin
SQPOLL kernel thread	on, idle timeout 5 s	zero-syscall steady state
IOPOLL	on for latency pools	no completion interrupts on the hot core
Multi-shot operations	read, stat, openat	one SQE serves repeated requests

Table 6: Ring geometry defaults (tunable per mount)

3.2 The bypass plane: SPDK and DPDK where every nanosecond is billed

For dedicated storage hosts, an optional **user-mode plane** takes the device away from the kernel entirely. The SPDK NVMe driver binds devices to a vfio-pci slot, and each engine shard owns one queue pair whose doorbell register is written directly from userspace - a doorbell write costs on the order of 100 nanoseconds, and there is no syscall, no interrupt unless requested, and no shared kernel block layer on the path. DPDK plays the same role for the replication network plane. The bypass plane is not the default: it forfeits the kernel page cache, unified device naming, and any POSIX consumer that is not LionFS-aware. The mount option selects the plane per pool, and the engine presents identical semantics above either substrate so that the benchmark protocol can run identically on both.

Plane	Per-op overhead	Semantics	Deployment
io_uring + interrupt CQ	approx. 0.3 microsec	full POSIX via VFS bridge	default, shared hosts
io_uring + SQPOLL/IOPOLL	approx. 0.1 microsec	full POSIX, pinned poll thread	latency-sensitive pools
SPDK poll mode	approx. 0.05 microsec	LionFS clients only	dedicated storage hosts

Table 7: Submission plane selection

3.3 The lock-free core

The engine is sharded by an injective hash of the file identity, so that a request's entire path - cache probe, extent resolution, allocation, submission - executes on one core with data structures that core alone owns. Cross-shard interaction is confined to two bounded queues: completed work drains to a completion dispatcher, and free-space returns flow through a multi-producer, single-consumer channel per device region. Path lookup is an **RCU**-protected read: dentries and inode cores are immutable snapshots, readers walk without locks, and a new generation is published by pointer swap with deferred reclamation after a grace period. Writers to the same file serialize through a **seqlock** - readers retry only when a writer interleaved, and writers themselves are rare because transactions batch. The sharding skeleton fits in a dozen lines and is the shape every hot-path structure in this RFC assumes:

```
// Per-core engine shard (simplified)
struct Shard {
    extent_cache: RcuCache<Lba, Extent>,    // owns this shard's hot set
    free_q: FreeQueue,                      // local chunk of free space
    ring: IoRing,                           // device submission handle
    tx: TxBuffer,                           // dirty pages awaiting commit
}
fn shard_of(fd: Fd, ino: Ino) -> usize {
    let h = splitmix64(fd.bits() ^ (ino.low() << 32));
    (h as usize) & (NUM_SHARDS - 1)         // power-of-two shards
}
```

3.4 Tiered and direct memory access: CXL and DMA

CXL-attached persistent memory enters the tier hierarchy as a byte-addressable cache and journal tier. Metadata leaves, the intent journal, and the dedup bloom filters are placed there first: the journal's fdatync collapses to a cache-line writeback with CLWB plus a fence, roughly two orders of magnitude cheaper than a flash flush, which transforms the fsync-heavy workload class. Device DMA is steered into CXL memory for read-modify-write payloads (parity deltas, cluster recompression) so the transform never round-trips through DRAM it does not need. The engine treats the tier as a first-class placement target with its own bandwidth accounting rather than as a block device, because treating PMEM as a disk is precisely the anti-pattern that wastes it. Where the platform exposes DSFC or CXL III shared devices, the same descriptor path issues direct device-to-device DMA transfers for replication, copying pools without hosting bytes in DRAM at all.

0 syscalls per steady-state operation (SQPOLL + registered buffers)	1 copy per read: the device DMA itself
--	---

4. Pillar II: Addressing, Namespace and File Specialization

4.1 The 128-bit volume address space

Every byte LionFS manages is named by a 128-bit volume address whose format is fixed at mkfs time and validated at mount. The width is chosen for reach with room to spare: a 128-bit namespace in 4 KiB units addresses 2 to the power of 140 bytes, four orders of magnitude beyond a yottabyte, so containers, checksum trees, and future replication formats never need a width migration. The layout composes a pool

of up to 16 million devices into volumes without widening hot structures, because the per-device fields are the ones that move:

Bits	Field	Meaning
127-112	volume_id (16)	subvolume / container selector
111-88	region (24)	stripe or band within the pool
87-64	device (24)	pool member (16.7M devices max)
63-0	device_lba (64)	per-device block address, 4 KiB units

Table 8: 128-bit volume address (logical), field layout

The deliberate conservatism is in the **packed extent record**, not the namespace. Extents are the single most numerous structure in any file system - a 10-terabyte streaming workload allocates them millions of times per hour - so their on-disk width is minimized to 16 bytes, one cache line holds eight, and a B-epsilon leaf holds hundreds. The trade is stated openly in Section 10: per-file and per-device encoded reach is exabyte-class rather than yottabyte-class, which is beyond every device on any roadmap, and the namespace layer above it carries the full width.

```
// Packed 16-byte extent record (little endian, bytemuck-Pod)
[ logical_start: u48 | physical_start: u48 | length: u24 | flags: u8 ]

flags: GRAN RAW ENC SHARED DEDUP ..reserved (3)
GRAN=0: all three fields count 4 KiB units (file max 1 EiB)
GRAN=1: all three fields count 64 KiB units (file max 16 EiB)
RAW: stored uncompressed | ENC: payload encrypted
```

4.2 Small files: inline in the metadata, tail-packed to the last byte

A file smaller than 4 KiB is stored entirely inside the B-epsilon leaf that holds its inode core - the payload is appended directly after the fixed fields as a variable length value, so reading the file is one metadata read and zero data-block reads, and creating it allocates zero data blocks. This is the mechanism the 1.x gap analysis demands: 4 KiB minimum per file disappears entirely, and small-file CRUD collapses to metadata I/O, which the RCU cache serves from memory in the common case. Because leaf values are variable-length, leaf packing would still waste the tail of the final leaf block; **dynamic tail packing** therefore co-packages the final partial record of one inode with the leading bytes of the next entry in the same leaf, with a 2-KiB leaf flush threshold that batches inode churn so a leaf re-write amortizes over many mutations. The 1.x spec's 256-byte inode becomes a 96-byte core in v3: identity, size, generation, flags, and either the inline payload length or the extent-tree root reference.

Offset	Field	Width	Notes
0	ino	16 B	full 128-bit inode number
16	mode / nlink / uid / gid	4 x u32	POSIX identity
32	size	16 B	u128, GRAN-aware
48	generation	8 B	bumped on every CoW rewrite
56	flags	4 B	INLINE, COMPRESSED, ENCRYPTED, DEDUP
60	extent_root / inline_len	6 B / 4 B	u48 tree ref, or payload length
64 or 68	inline payload	0-4032 B	only when INLINE flag set

Table 9: Inode 2.0 core layout (variable-length leaf value)

4.3 Large files: the B-epsilon extent index

Above the inline threshold, extents live in a per-inode **B-epsilon tree**, the write-optimized structure of Bender and colleagues, whose internal nodes have large fanout and whose leaves are oversized buffers that absorb writes and flush lazily in sorted runs. For a file system this shape is precisely what the workload wants: allocations and truncations are leaf appends that amortize one internal-node rewrite across hundreds of mutations, while reads in the steady state hit the in-memory copy of the hot leaf and cost one comparison per level. The 1.x spill B-tree rewrites tree nodes on nearly every insert - which is a measurable fraction of why rand4k-write sat at +0.8 percent - and the B-epsilon leaf turns that into an append. Leaves are padded to 25 percent free space on flush so hot files do not split on every rewrite, and the flusher coalesces adjacent extents before writing, which is the second half of how the 8192-to-8 fragment result extends to arbitrary scales.

Property	Plain B-tree (1.x)	B-epsilon (2.0)	HAMT (namespace only)
Insert cost	rewrite internal nodes often	leaf append, lazy flush	O(1) rehash copy
Range reads	excellent	excellent (sorted runs)	poor - no ranges
Write amplification	high on random	amortized, padded leaves	n/a
Memory (hot set)	node cache	leaf cache, 2-4 KiB leaves	pointer table 1/8 size
Role in 2.0	retired	extent + metadata index	inode number space

Table 10: Extent index options, and why B-epsilon wins for LionFS

The **HAMT** is not wasted by this choice: it is the inode-number space. 128-bit inode keys hash into a persistent HAMT whose depth grows only when populations demand it, giving O(1) amortized lookup for name resolution results and stable 16-byte keys for the replication plane - while range-scannable data

stays in B-epsilon trees where range queries actually exist. Directory blocks, the third namespace structure, remain hash-split B-epsilon leaves of (name, ino) pairs, replacing the 1.x per-directory B-tree with the same write-optimization argument as extents.

5. Pillar III: Reliability, Self-Healing and Crash Recovery

5.1 Crash consistency: redirect-on-write plus intent journal

LionFS 2.0 keeps the architecture 1.x proved correct and hardens the ordering. Every mutation is redirect-on-write: new extents and new tree nodes are written to fresh locations, and visibility flips atomically when the transaction's root pointers move. The intent journal precedes data: a transaction first writes an **intent record** enumerating every extent, cluster, and tree node it will touch and `fdatasyncs` it; data and metadata then stream to media; a single tagged 4 KiB **commit record** closes the transaction; and a background checkpoint later swaps the tree roots under a generation counter and rewrites all three superblock copies. Recovery is therefore $O(\text{journal})$, not $O(\text{volume})$: mount replays the intent log, rolls forward fully-committed transactions, discards partial ones, and the file system is consistent in the time it takes to read the log - there is no fsck mode at all, and none is needed, because no overwrite ever makes on-disk state ambiguous.

```
// Commit ordering (the only ordering that matters)
1. tx.intent <- { extents[], nodes[], roots[] } ; fdatsync(journal)
2. write data extents, cluster payloads, parity deltas // unordered
3. journal.commit_record <- { tx.id, crc32c, generation } // single 4 KiB
4. fdatsync(journal) ; flush device cache (FUA on ring)
5. checkpoint (background): swap roots, rewrite SB0/SB1/SB2
6. release old-generation extents to free-space queues
```

Torn writes are bounded by construction: the commit record carries a CRC and the generation counter, so a partially-written record is simply an absent record, and a half-written data extent behind a valid commit is detected by its checksum and repaired from parity rather than trusted. The 1.x discovery that incremental parity is not idempotent under replay is carried forward explicitly: journal replay recomputes parity rows in full, and only the live path uses the incremental RMW.

5.2 Dual-speed checksums and continuous scrubbing

Integrity verification is split by temperature because checksum strength and checksum speed trade linearly. Hot data blocks carry **xxHash64** - on the order of 20 GB/s per core with AVX-2 multi-buffer -

verified on every read completion. Cold data and every compression cluster carry **BLAKE3** with its tree-shaped, SIMD-parallel digest: cryptographic strength at multi-GB/s per core, so the cluster that is already being decompressed is verified essentially for free. The 1.x gap where compressed inodes detected corruption only via zstd decode failure is closed: every cluster record pairs with a 16-byte BLAKE3 tag in the checksum tree, at 0.4 percent metadata overhead. The scrubber walks the checksum tree continuously, rate-limited to a configurable share of device bandwidth with defaults at 5 percent foreground-free, targeting one full pass per week per pool, and its findings feed the bad-blocks tree the 1.x implementation already maintains.

Primitive	Throughput class	Applied to	Property
xxHash64	approx. 20 GB/s/core	hot 4-64 KiB pages	collision-safe for bit rot; fast
BLAKE3	multi-GB/s/core, tree-parallel	cold pages, clusters, journal	cryptographic, keyed mode available
CRC32C	hardware instruction	commit records, superblocks	torn-write detection
GF(256) P/Q	table + planned SIMD	parity pools	self-heal math, 1.x engine retained

Table 11: Integrity primitives and where each applies

5.3 Autonomous repair

Detection without repair is just surveillance. When a read or the scrubber finds a checksum mismatch, the offending block is quarantined in the bad-blocks tree, the scrubber allocates a fresh extent through the normal per-core allocator, reconstructs the data from P/Q parity (or from a mirror), writes it with a fresh checksum, and swaps the extent reference inside a first-class transaction - the same intent-log machinery as any other mutation, so a crash mid-repair heals into either the old or the new copy and never into neither. Repair is therefore **autonomous**: no operator action, no unmount, no fsck run. Pools without redundancy mark the block and report the loss event to the health bus rather than pretending. The scrub schedule is adaptive: pools with observed bit-rot history scrub hot regions first, and the scheduler prefers device idle windows so the healing path itself respects the latency SLO of foreground traffic.

5.4 The recovery state machine

Recovery is a five-state machine executed by the mount path, and every transition has one obligation: make the smallest change that restores a provably consistent view, then get out of the way. The mount reads all three superblocks, selects the copy with the highest CRC-valid generation, and enters intent replay; the journal is walked from its sequence number, and each closed transaction - one whose commit record survived - is rolled forward by replaying its enumerated extent and node writes idempotently,

while open transactions are discarded whole, which is always safe because redirect-on-write means their partial writes landed in blocks no live tree references. Replay then checkpoints immediately, so the journal can be reset before the first user operation, and the bad-blocks and zone tables reconcile against device reports before the file system goes writable. The whole path is exercised by the failure-injection harness specified in Section 9.5; a crash at any instruction boundary must land in a state this machine converges, and that property is tested, not asserted.

1. PROBE: read SB0/SB1/SB2, choose highest generation with valid CRC32C.
2. REPLAY: walk intent log from journal_seq; roll forward committed, discard open.
3. CHECKPOINT: swap roots, rewrite superblocks, reset the journal.
4. RECONCILE: merge bad-blocks and ZNS zone tables with device-reported state.
5. WRITABLE: open rings, start shards, begin accepting submissions.

6. Pillar IV: Hardware-Aware Tiering and Alignment

6.1 Media policies

The allocator is a policy engine over device classes, and every policy is a function of geometry the engine probes rather than guesses: NVMe Identify for namespace granularity, zone size and optimal write size for ZNS, log-physical sector size and smallest-allocation hints for SATA and SAS, and CXL memory device latency classes for the PMEM tier. For **ZNS** host-managed drives the write path becomes zone-append: the engine submits appends with a 64-bit write pointer token per zone, the device places the write wherever its media wants, and the returned offset is recorded in the extent - write amplification on sequential fills drops to approximately 1.0 and the flash translation layer is bypassed entirely. For **SMR** drives the allocator confines each actively-written file to one sequential band, the scheduler batches cross-band reclaims into elevator sweeps at device-idle windows, and random writes to SMR-host-managed pools are rejected at open time with an explicit error rather than silently degrading - an honest failure 1.x-style tooling can surface. The **PMEM tier** absorbs the intent journal, metadata leaves, and dedup filters as Section 3.4 specified, sized so that a full journal checkpoint is a CXL writeback, not a device flush.

Media	Placement policy	Write path	Alignment unit
NVMe ZNS	one file per zone until 85% full	zone append + write pointer token	zone size, 2-4 GiB
NVMe / SSD	locality clusters, speculative extents	queued writes, FUA at commit	4-16 KiB probed
HDD SMR	band-confined sequential	elevator batches, idle reclaim	band, 256 MiB typical
HDD PMR	outer-LBA-biased free-space runs	merged large writes	1-4 MiB merge window
CXL PMEM	journal, leaves, bloom filters	CLWB + fence	cache line, 64 B

Table 12: Media policy matrix

6.2 Universal alignment guarantees

Alignment is enforced at the three places misalignment can be introduced. At mkfs, the superblock records probed logical sector size, physical sector size, and optimal I/O size, and free-space regions are snapped to the optimal I/O boundary. At allocation, every extent request is rounded up to the device's **page cluster class** - 4, 16, or 64 KiB - with the rounding accounted as padding in the extent record's GRAN mode rather than as file size, so user-visible sizes never lie. At submission, I/O descriptors are split and merged so each device command is a multiple of the probed optimal size and offset-aligned to it; the engine keeps the 1.x debug counters that measure alignment violations, because a guarantee you do not measure is a hope. Misaligned requests can still occur from hostile unaligned user buffers - those are served through a bounce-buffer slow path with an explicit perf counter, visible in the health bus, never silently copied.

7. Pillar V: Compression and Deduplication Pipeline

7.1 Tiered adaptive compression

The 128 KiB cluster scheme from 1.x is the substrate, and its measured behavior drives the 2.0 upgrades. A corpus of 40 percent repeating records, 35 percent dictionary text, and 25 percent incompressible bytes compressed 2.90x at zstd level 3 while writing at 407 MiB/s on a 2-vCPU container, and the level sweep showed the honest cliff: level 9 buys 0.08x more ratio for 6.8x the CPU. The 2.0 pipeline therefore adapts per inode: the first two clusters written measure compressibility and latency, and the policy engine pins the file to a tier - **LZ4** for anything that must sustain line-rate writes (it decompresses at several GB/s per core and compresses fast enough to stay on the write path), **zstd level 3** for warm bulk, **zstd level 9-plus on the cold tier** where write rate is irrelevant and ratio compounds across terabytes. Hardware acceleration is a first-class path, not an afterthought: Intel QAT devices compress and checksum entire

clusters in flight, AVX-512 vectorizes the codec kernels themselves, and the transform pipeline selects among software, SIMD, and QAT backends per submission based on availability and queue depth, with the selection itself a measured decision recorded in the health bus.

Tier	Codec	Write path target	Integrity
Hot	LZ4 block	inline, no added latency	xxHash64 per page
Warm	zstd level 3	1-3 GB/s per core	BLAKE3 per cluster
Cold	zstd level 9+ / QAT	background, ratio-first	BLAKE3 per cluster
Raw fallback	none	incompressible clusters	xxHash64, RAW flag set

Table 13: Compression tier policy

The worst 1.x behavior - a random 4 KiB write into compressed data costing a full 128 KiB decompress-splice-recompress - is answered with a **punch-through escape hatch**: when the policy engine observes a third RMW against the same cluster, the cluster is transparently decompressed into raw extents, its ClusterTree entry is retired, and subsequent random writes hit the plain extent path. Write amplification on the transition is paid once instead of unboundedly. The reverse direction exists too: clusters that go cold and unmodified for a scrub cycle are re-compressed into the cold tier during idle windows, because backup pools deserve the compaction without the hot path ever paying for it.

7.2 Inline deduplication with content-defined chunking

Cold and backup pools deduplicate inline. Chunks are cut with **FastCDC-style content-defined chunking** - expected size 8 KiB, min 2 KiB, max 32 KiB - so insertions and deletions shift cut points only locally and identical content chunkes identically regardless of file alignment. Each chunk is hashed (BLAKE3-128), and the hash is probed against a three-level index: a small in-RAM bloom filter over the whole pool, a bounded LRU of hot chunk hashes, and the on-disk hash tree consulted only when the filters say maybe. Duplicate hits append a reference to the existing chunk extent under the refcount tree the 1.x format already defines; misses write the chunk once. The memory budget is explicit and bounded: the bloom filter and hot cache together default to 0.1 percent of pool size in RAM (1 GB per TB), and the honest consequence - a cold duplicate costs one hash-tree walk - is recorded in Section 10 rather than hidden. Inline dedup is disabled on hot pools by default because deduplication randomizes layout, and Section 10 prices that trade too.

```
// Cluster write decision (per 128 KiB cluster)
match compress(data) {
    Ok(payload) if ratio >= 1.2 =>
        write_cluster(payload, BLAKE3, tier)
    _ => write_raw_extents(data, xxhash64)    // RAW flag
}
on 3rd rmw_hit(cluster): decompress_to_extents(cluster) // punch
```

7.3 Ratio accounting and the honesty rule

How a compression ratio is measured decides whether it is a fact or a story, so the 2.0 specification inherits the 1.x accounting rule verbatim: ratios are computed from the allocator's own free-space accounting - logical bytes written versus physical blocks actually consumed - never from compressor return values, and never from du against a mounted view that might be folding in other effects. The 1.x result of 2.90x on the deliberately-mixed corpus (40 percent repeating records, 35 percent dictionary text, 25 percent PRNG bytes, measured from the bitmap) is the template: the corpus is hostile on purpose, the measurement is at the allocation layer, and the raw outputs are checked in beside the command that produced them. Phase P5's exit criteria add the same discipline for hardware offload: QAT-assisted throughput is reported per backend, per queue depth, and against the software path on the same host in the same run, because a hardware number without a software control is exactly the kind of claim the honesty rule exists to prevent. Dedup ratios get the same treatment at the chunk layer - bytes referenced versus bytes stored - with the RAM cost of the index reported alongside, so the review board can price the trade rather than admire the number.

8. Deliverable A: Core Architecture Blueprint

The blueprint is presented as three planes. The host plane owns submission: applications, the VFS bridge for POSIX compatibility, and per-application io_uring rings with registered buffers. The core engine is where every LionFS semantic lives, sharded per core, with RCU-protected lookups, the B-epsilon extent index, the transaction engine, per-core allocators, the transform pipeline, and the policy, scrub, and background workers. The storage plane fans work out to io_uring device rings or SPDK queue pairs, through the RAID/erasure engine, down to the four media classes. Requests flow down; completions flow up as CQEs; nothing crosses a plane synchronously. Figure 2 is the whole system at one glance, and the sections that follow give the field-level contracts for what its boxes persist.

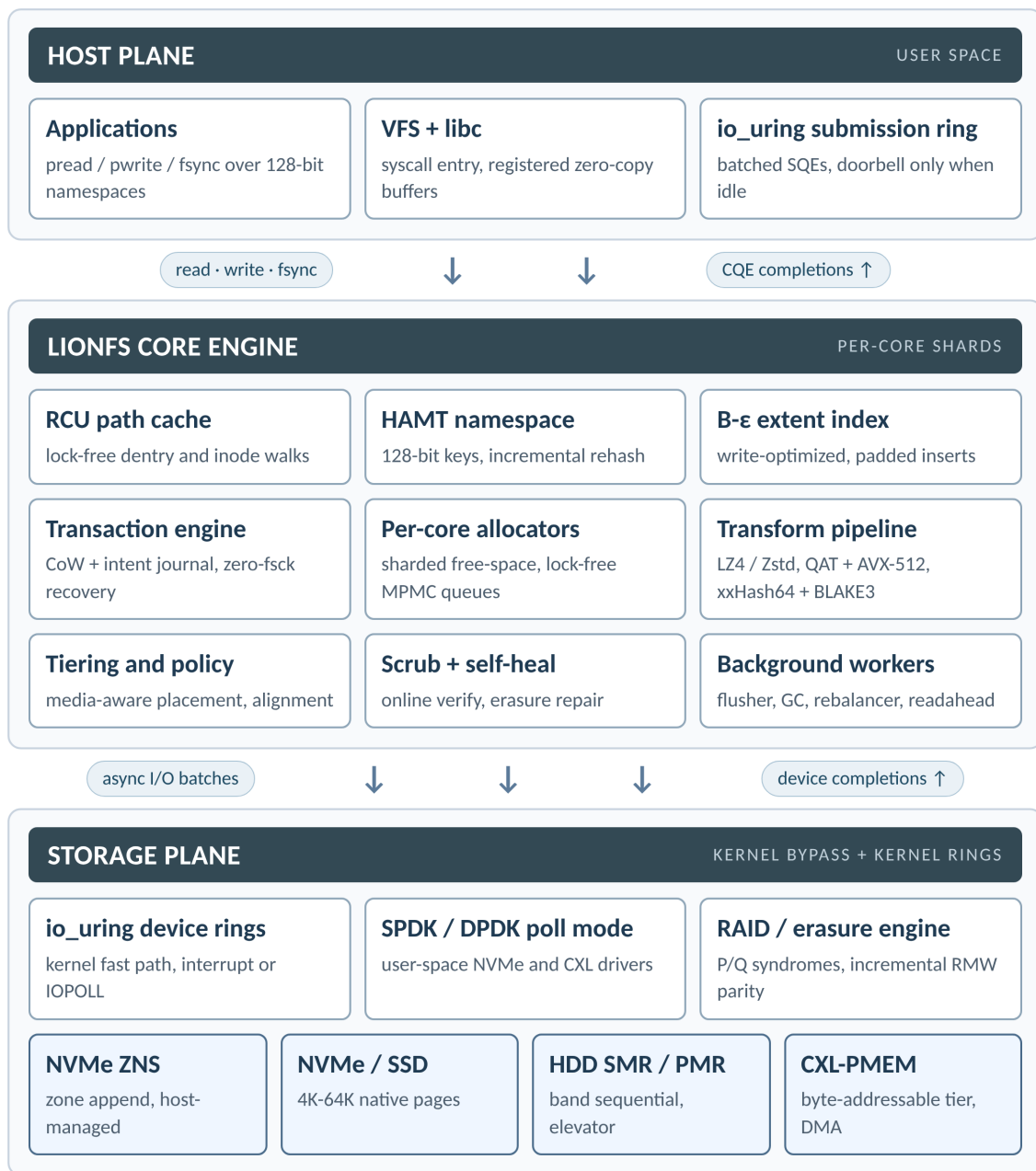


Figure 2: LionFS 2.0 core architecture - three planes, per-core shards

8.1 On-disk layout

The volume is laid out as a linear strip per device: three superblock copies (LBA 0, LBA 1, and the final aligned block), a circular intent journal, the metadata zone (B-epsilon leaves and internal nodes, HAMT nodes, checksum tree, ClusterTrees, free-space runs, region map), and the data zone of variable-length extents, compression clusters, and parity stripes. Everything above the superblock is addressed by tree references rather than fixed offsets, so the zones grow and shrink by allocation, and every structure carries its own checksums. Figure 3 shows the strip and the magnified records the rest of this section

defines precisely.

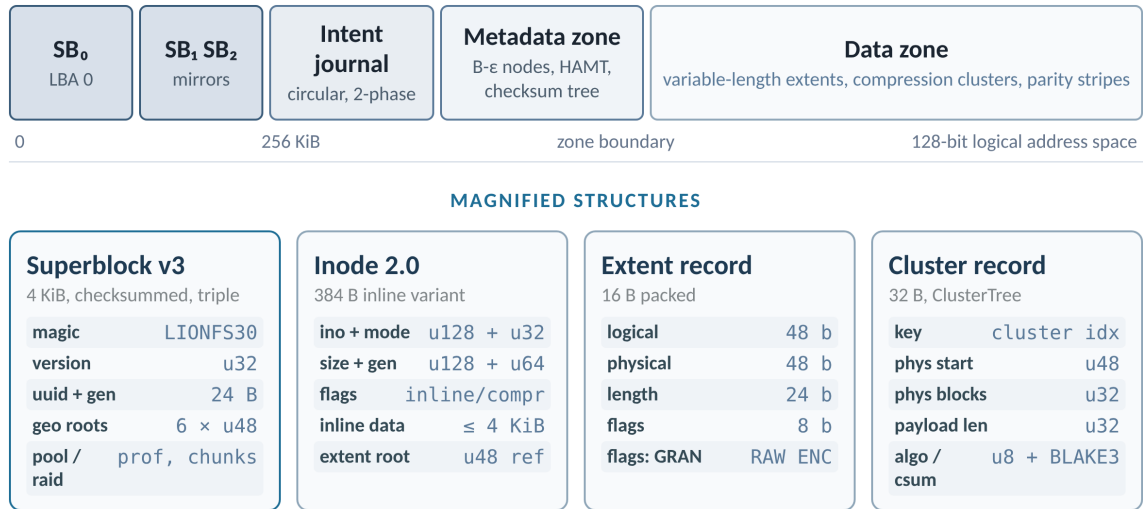


Figure 3: v3 on-disk layout and packed record formats

Offset	Field	Format	Notes
0x00	magic	8 B	LIONFS30
0x08	version	u32	3; mount gates: older readable, newer refused
0x0C	page_cluster	u8	log2 of allocation unit class
0x10	uuid	16 B	volume identity
0x20	generation	u64	bumped at every checkpoint
0x28	feature_flags	u64	COMPRESS, DEDUP, ENC, ZNS, PMEM tier
0x30	roots: bepsilon, hamt, csum, cluster, free, region	6 x u48	tree root LBAs
0x54	journal_start / journal_len / journal_seq	u48 + u32 + u64	circular intent log
0x68	pool: profile, chunk_size, device_count	u8 + u32 + u24	RAID topology
0x74	geometry: log_sec, phys_sec, opt_io	3 x u32	probed at mkfs
0x80+	counters + padding + crc32c	rest	last 4 B of the block

Table 14: Superblock v3 (4 KiB, CRC32C, written to SB0/SB1/SB2)

8.2 Allocation structures

Free space is represented twice, on purpose. The authoritative form is a per-region **free-space tree** of extent holes keyed by physical LBA, checkpointed with the volume; the working form is the per-shard in-memory queue of allocation runs that Section 3 described, refilled by batched tree reads and reconciled at checkpoint. The 1.x bitmap is retained only as a mkfs-time bootstrap format, because a full-volume bitmap at 128-bit LBAs would be absurdly large while a per-region extent tree is proportional to fragmentation, not capacity. Allocation policy attaches to each run: media class, alignment unit, zone or band affinity, and locality hints derived from the writing inode's last extent - the exact locality machinery 1.x proved out, moved into the run descriptor.

Structure	Form	Owner	Consistency
Free-space tree	B-epsilon, holes by LBA	checkpointed volume state	generation-gated
Shard free queue	in-RAM runs, bounded	one per core shard	reconciled at checkpoint
Region map	flat table in SB zone	mkfs + grow operations	triple-redundant with SBs
Zone table (ZNS)	write-pointer tokens	storage plane	recovered from device report
Bad-blocks tree	refcounted quarantine	scrubber	transactional, 1.x structure kept

Table 15: Allocation structures and their roles

8.3 Journaling and copy-on-write trees

Six trees form the metadata spine, and all of them are copy-on-write with generation gating. The B-epsilon extent/metadata index and the HAMT namespace were specified in Section 4. The **ClusterTree** persists the 32-byte cluster records of Section 7, keyed by cluster index, rooted per inode. The **checksum tree** maps every persistent block and cluster to its xxHash64 or BLAKE3 tag. The **free-space tree** and **refcount tree** close the loop for allocation and dedup. The intent journal is not a tree but a circular log of transaction records, and it is the only structure written in place - with the tag-and-generation discipline of Section 5 making in-place writes safe. Tree roots move atomically at checkpoint; between checkpoints the journal is the bridge, and recovery replays it forward as Section 5.1 specifies.

Tree	Key	Value	Fanout shape
B-epsilon index	extent logical start	16 B packed extent	high internal fanout, fat leaves
HAMT namespace	128-bit inode number	inode core + inline payload	hash trie, 5-bit nibbles
ClusterTree	cluster index u64	32 B cluster record	B-epsilon leaf per inode
Checksum tree	physical LBA	8 B xxHash64 / 16 B BLAKE3	B-epsilon, cold-tier flushed
Free-space tree	hole start LBA	hole length	B-epsilon per region
RefCount tree	chunk hash	u32 refs + extent	B-epsilon, dedup pools only

Table 16: The metadata spine at a glance

9. Deliverable B: Read/Write Lifecycle Walkthrough

This section traces one 1 MiB read and one 1 MiB write from the application call to the physical media, with per-step costs stated against the budget of Table 4. The phases are the same four shown in Figure 1; here they are expanded to the step level, and the tables are the normative reference the implementation phases P0-P3 will be validated against. Costs are per-operation CPU work unless marked device-bound, and they are targets with the same status as the exit criteria of the roadmap: measured results will be published beside them, not instead of them.

9.1 Reading 1 MiB (warm file, compressed pool)



Figure 4: Read lifecycle - five phases, per-phase cost targets

#	Step	Component	Cost target
1	pread(fd, buf, 1 MiB) enters the VFS bridge; fd resolves to inode via RCU dentry cache	host plane	0.3 microsec
2	request becomes 8 cluster SQEs on the file's ring; buffers were registered at open	io_uring	0 (no syscall)
3	shard resolves 128-bit offsets to extents by shift-split; extent cache hit	core, per-shard	1 microsec total
4	cache miss path: async B-epsilon leaf read, RCU publish, retry once	core + metadata zone	device-bound, overlapped

#	Step	Component	Cost target
5	compressed clusters fetched whole; decompression overlapped with next fetch (LZ4 tier)	transform pipeline	approx. 1 microsec/4 KiB, hidden
6	parity pools fan page reads through the erasure engine	storage plane	device-bound
7	device DMA lands pages in the registered user buffer directly	device	zero CPU copies
8	xxHash64 verified per page against the checksum tree; mismatch goes to repair	completion path	0.1 microsec/4 KiB, deferred
9	CQEs reaped without syscalls; readahead recorder updates Markov table	completion path	0.2 microsec

Table 17: Read walkthrough, step by step

9.2 Writing 1 MiB (transactional, RAID5 pool)



Figure 5: Write lifecycle - CoW transaction pipeline with crash points

#	Step	Component	Cost target
1	dirty 4 KiB pages stage in the shard's tx buffer; batch flush at 64 KiB or fsync	core, per-shard	amortized
2	allocator reserves speculative aligned extent from the shard run; GRAN chosen	allocator	lock-free, 0.2 microsec
3	intent record (extents, nodes, roots touched) written to journal and fdatasynced	intent journal	1 device flush, group-committed

#	Step	Component	Cost target
4	clusters compressed (tier policy), checksums computed, parity deltas folded	transform + erasure	1-3 GB/s per core
5	data + metadata + parity submitted in batches; FUA on the final ring	storage plane	device-bound
6	commit record written: one tagged 4 KiB journal block closes the transaction	intent journal	1 device flush
7	completions update deferred checksum entries; CQEs reaped	completion path	0.1 microsec/4 KiB
8	background checkpoint: roots swap under generation, 3 superblocks rewritten	flusher worker	off the latency path
9	old-generation extents released to free queues; GC coalesces runs	GC worker	off the latency path

Table 18: Write walkthrough, step by step

9.3 Variants that matter

Two variants complete the picture because they are the ones the workload matrix grades. An **inline small-file write** of 800 bytes never touches the data zone: the payload is appended to the inode's B-epsilon leaf, the leaf flush batches it with neighboring inode churn behind a 2 KiB threshold, the intent journal protects the leaf rewrite, and the read path resolves it in one metadata read that the RCU cache usually serves from memory - the whole lifecycle is metadata I/O, which is how the small-file CRUD target of a 10x improvement over 1.x is meant to be hit. A **ZNS append write** replaces steps 2 and 5: no extent is pre-reserved, the storage plane issues zone-append with the zone's write-pointer token, the device returns the placed offset, and the completion path writes the extent record then - write amplification approaches 1.0 and the FTL is a spectator. Both variants are first-class citizens of the same transaction machinery, not bolted-on special cases; that is the point of specifying the pipeline once and profiling every path against it.

9.4 fsync, group commit, and the flush path

The fsync path is where the durability SLO of Section 2.4 is won or lost, and the mechanism is group commit through the intent journal. When a writer calls fsync, its shard's transaction joins the next journal batch rather than forcing its own: the flusher closes the batch - whose size is bounded by time and bytes, not by writer count - writes the concatenated intent records in one journal append, fdatsyncs once, streams the batch's data and parity with FUA on the final ring submission, and writes one commit record per batch. Sixty-four concurrent fsyncers therefore cost one device flush per batch window instead of sixty-four, which is the entire difference between a transactional file system and a throughput benchmark casualty. The 1.x transaction manager already batches between fsyncs; the 2.0 change is that the batch policy is explicit, measured, and owns an SLO. Writers that need isolation from group semantics - a batch

that fails rolls back every member - get a private transaction through the same journal at the cost of their own flush, chosen at fsync time, not silently.

```
// Group commit (flusher worker, per batch window)
batch = collect(tx_ready, max_ms=5, max_bytes=1 MiB)
journal.append(intents(batch)) ; fdatsync(journal) // 1 flush
rings.submit(data(batch) + parity(batch), fua=LAST) // device flush 2
journal.append(commit_record(batch)) // close
for w in batch: complete(w) // CQEs to every fsyncer
```

9.5 Failure injection and proof obligations

Crash consistency claims are cheap; this RFC's are backed by a failure-injection harness with the same standing as the benchmark protocol. The harness runs the transactional workload against a dm-error or device-mapper fault-injected pool, killing the engine at instruction-level and I/O-boundary checkpoints - before intent, between intent and data, mid-data, after data before commit, after commit before checkpoint - and after every kill it mounts and asserts the invariants: no visible transaction is partial, no committed transaction is absent, every readable block matches its checksum or is quarantined, and the free-space accounting reconciles to the block map. The obligations are exactly the two crash points annotated on Figure 5 plus every boundary between them; the property tests in the 1.x tradition (proptest round-trips for the B-tree, parity equivalence for RAID) extend to the B-epsilon leaves and the punch-through transition. A phase that touches the write path ships with new injection cases; a phase that cannot say which cases it added has not finished its design review.

- Kill points: pre-intent, post-intent, mid-data, post-data, post-commit, mid-checkpoint.
- Invariants: transactional atomicity, checksum validity, free-space reconciliation.
- Harness: dm-error injection plus ptrace-scheduled process kills, 500 cycles per phase.
- Sign-off: new write-path code requires new passing kill points, listed in the phase report.

10. Deliverable C: Trade-off Analysis

Every design decision above is a position taken in a known tension. This section states each one as an engineering position: the classical tension, the naive resolution and its cost, the mechanism LionFS 2.0 uses, and the residual risk that remains after the mechanism - because a trade-off you claim to have eliminated is a trade-off you have merely stopped measuring. The two tables are the contract the review board should hold the implementation to: every residual risk row has a counter in the health bus or the benchmark protocol.

Tension	Naïve resolution and cost	LionFS 2.0 mechanism	Residual risk
CoW fragmentation vs in-place write speed	In-place updates: fast overwrites, corruptible metadata, no snapshots	RoW everywhere + speculative sequential extents + GC re-sequencing (1.x measured 8192 to 8 fragments)	Transient fragmentation under mixed random+sequential files until GC catches up
Checksum insert cost vs integrity	Skip checksums on the write path: silent corruption window	Deferred verify on completion + B-epsilon leaf appends (1.x measured insert at approx. 45% of write cost)	Corruption detectable only after completion; quarantine window is one flight of I/O
Memory footprint vs lookup latency	Cache everything: latency vanishes, RAM scales with capacity	Per-shard extent cache (64 B/entry) + RCU leaves; 99% hit target on hot sets	Cold random reads pay one metadata device read on miss
Kernel bypass vs generality	SPDK everywhere: fastest, but un-POSIX, un-shared devices	io_uring default + SPDK opt-in per pool with identical semantics above both	Two planes to validate; bugs can hide in the less-used one
Dedup RAM vs dedup savings	Full in-RAM index: instant dedup, RAM grows with pool	Bounded bloom + hot LRU + on-disk hash tree (0.1% of pool in RAM)	Cold duplicate costs one tree walk; dedup off on hot pools by default

Table 19: Core structural trade-offs

Tension	Decision	Rationale	Residual risk
128-bit vs 256-bit addressing	128-bit namespace; 48-bit packed per-device fields	No medium approaches 2^{40} blocks; wider keys split cache lines and slow hashing	16 EiB coarse-mode file cap; region indirection for exotic future pools
Compression ratio vs CPU latency	Tiered LZ4 / zstd-3 / zstd-9+ with QAT and punch-through	1.x measured the cliff: +0.08x ratio for 6.8x CPU beyond level 3	Cold-tier re-compression passes consume idle bandwidth
Inline small files vs metadata write amplification	Leaf tail packing with 2 KiB flush threshold	1 metadata read per file, 0 data blocks; churn batches into leaf rewrites	Sustained overwrite of one small file rewrites its leaf (bounded by batching)
Cluster RMW vs per-block compression	128 KiB clusters with punch-through escape	Clusters are what make compression save real space (2.90x measured); escape bounds the RMW tax	Two random writes per cluster still pay full RMW once
Strong checksums vs throughput	xxHash64 hot, BLAKE3 cold + per-cluster	BLAKE3 rides the decompression already happening; xxHash64 rides completion	Mixed-temperature files pay both once at tier transitions
Zone append vs deterministic placement	ZNS appends with completion-time extents	WAF approx. 1.0, FTL bypassed; device places writes optimally	Extent map fills after the fact; recovery trusts device zone reports

Table 20: Format and encoding trade-offs

The pattern across all twelve rows is the same discipline: choose the side of the tension that preserves the measured, reproducible property (integrity, honesty of layout, ratio) and then spend mechanism - deferral, batching, escape hatches, background workers - to claw back the performance the naive choice would have given away for free. Where the mechanism cannot fully close the gap, the residual risk is named and made observable rather than argued away. That is the same engineering stance the 1.x benchmark document took when it shipped readahead default-off and called rand4k-write flat, and it is the stance the 2.0 phases will be graded on.

11. Implementation Roadmap

The roadmap maps the target architecture onto the existing codebase as seven phases, each a revertible commit series with exit criteria expressed as measurements, following the 1.x program's ground rules: build and test green before and after every phase, interleaved A/B medians for every claim, raw outputs checked in, and no number without the command that produced it. Phase ordering is dependency-driven: the substrate that changes where code runs (P0, P1) precedes the format that changes what code stores (P2, P3), which precedes the media and pipeline features that assume both (P4, P5, P6).



Figure 6: P0-P6 roadmap with per-phase exit criteria

Phase	Scope (1.x modules touched)	Exit criteria (measured)	Primary risk
P0	fuse/mount, disk/block_io, tools/ioperf	io_uring daemon mounts; fio on real NVMe at 85% line rate; FUSE compat green	Ring edge cases under crash
P1	cache/*, allocator queues, new shard module	perf lock shows zero global hot-path locks; rand4k +50% vs P0	Correctness under seqlock races
P2	ondisk/*, specifications, tools/upgrade	v3 mkfs/mount/upgrade; v2 images mount read-only; round-trip property tests	Upgrade tool fidelity at scale
P3	btree to B-epsilon, integrity deferred	rand4k-write 2x P0; checksum insert under 5% of write cost	Leaf flush heuristics tuning
P4	allocator policies, disk/geometry	ZNS pool WAF below 1.1; SMR band reclaims in idle windows; alignment counters zero	Device quirk matrix breadth
P5	fs/compression, file/cluster, dedup	Cold tier 2.5x ratio at 3 GB/s with QAT; punch-through visible in counters	QAT availability and licensing
P6	pool/replication, worker/scrubber, SPDK plane	Unattended bit-rot repair in 3-device soak test; CXL journal fdasync at CLWB cost	Bypass plane hardware variance

Table 21: Phase scope, exit criteria, and primary risks

11.1 Normative references

1. J. Axboe. io_uring and liburing interface documentation. Linux kernel sources, Documentation/block/io_uring.rst; github.com/axboe/liburing.
2. SPDK: Storage Performance Development Kit documentation. spdk.io.
3. INCITS. Zoned Host Commands (ZBC) and Zoned Device Commands (ZAC); NVM Express Zoned Namespace Command Set, NVMe 2.0.
4. M. A. Bender, R. Farach-Colton, J. T. Fineman, Y. R. Kuszmaul, J. Nelson. Cache-oblivious streaming B-trees. SPAA 2007.
5. P. Bagwell. Ideal Hash Trees. EPFL Technical Report, 2001.
6. J. O'Connor, S. Neves, J.-P. Aumasson, L. Perrin, et al. The BLAKE3 cryptographic hash function. blake3.io, 2020.
7. Y. Collet. xxHash - extremely fast non-cryptographic hash algorithm. github.com/Cyan4973/xxHash.
8. W. Xia, Y. Jiang, D. Feng, F. Douglass, G. Gibson, Y. Shilane. FastCDC: a fast and efficient content-defined chunking approach. USENIX FAST 2016.
9. Btrfs documentation: compression, extents, subvolumes. btrfs.readthedocs.io.

10. OpenZFS. On-disk specification and architecture reference. openzfs.github.io.
11. CXL Consortium. Compute Express Link (CXL) 3.0 Specification, 2022.
12. P. E. McKenney and J. Walpole. What is RCU? LWN.net, 2007.
13. LionFS 1.x repository: [docs/benchmarks.md](#),
[specifications/{superblock,inode,extents,transactions}.md](#), [src/file/cluster.rs](#). This submission's
baseline.